ACLIS Pest Gate — Distill + Field-aug + EMA + QAT

3-class gate: **`leaf`** / **`pest`** / **`others`**.

Adapted from the Leaf Gate alt recipe (`aclis_leaf_gate_alt_distill_qat.ipynb`):

	Leaf Gate (alt)	This notebook (Pest Gate)
Classes	leaf, not_leaf	**leaf, others, pest**
Data	Kaggle leaf / non-leaf	leaf + non-leaf + **PlantVillage pest**
Student	TinyLeafGate (~29k)	**Same topology**, 3-way head (`TinyPestGate`)
Teacher	MobileNetV3-Small	same
Recipe	KD + MixUp + EMA + field augs + QAT	same

Dataset (already built locally):

`Ikmal/Leaf/Pest Gate Model/leaf_pest_others_dataset/{train,val,test}/{leaf,others,pest}/`

- `leaf` ← leaf_noleaf `Leaf`
- `others` ← leaf_noleaf `Non_Leaf`
- `pest` ← `aclis_ready_plantvillage_dataset/*/pest`

Deploy artifact:

`pest_gate_output/aclis_pest_gate_96x_full_int8.tflite`

0 — Mount Drive & install packages

[1] ✓ 18s
```
from google.colab import drive
drive.mount('/content/drive')
```
```
Mounted at /content/drive
```

[2] ✓ 38s
```
# Same stack as leaf-gate alt notebook
!pip install -q tensorflow==2.20.0
!pip install -q onnx==1.17.0
!pip install -q onnx-simplifier onnxruntime
!pip install -q flatbuffers torch torchvision
!pip install -q pillow
```
```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.0/16.0 MB 100.6 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19.2/19.2 MB 97.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.7/2.7 MB 101.3 MB/s eta 0:00:00
  Building wheel for onnx-simplifier (pyproject.toml) ... done
```

[3] ✓ 38s
```
!pip install -q onnx2tf==1.25.2 --no-deps
!pip install -q sng4onnx
!pip install -q onnx-graphsurgeon --index-url https://pypi.ngc.nvidia.com
!pip install -q simple-onnx-processing-tools
!pip install -q nvidia-pyindex
```
```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 145.6/145.6 kB 8.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 440.0/440.0 kB 25.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 42.1/42.1 kB 2.9 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
  Building wheel for nvidia-pyindex (setup.py) ... done
```

[4] ✓ 8s
```
import tensorflow as tf
import torch

print('✅ TensorFlow :', tf.__version__)
print('✅ PyTorch    :', torch.__version__)
print('✅ CUDA       :', torch.cuda.is_available())
```
```
✅ TensorFlow : 2.20.0
✅ PyTorch    : 2.11.0+cu128
✅ CUDA       : True
```

1 — Config

Outputs go to a dedicated `pest_gate_output/` folder so leaf-gate artifacts are never overwritten.

[5] ✓ 0s
```
import os
from pathlib import Path

DRIVE_ROOT = '/content/drive/MyDrive'

# Prefer a prebuilt 3-class zip on Drive; else build from leaf_noleaf + plantvillage pest
PREBUILT_ZIP = f'{DRIVE_ROOT}/leaf_pest_others_dataset.zip'
LEAF_NOLEAF_ZIP = f'{DRIVE_ROOT}/leaf_noleaf_dataset.zip'
# PlantVillage-ready tree (must contain train/val/test/pest)
PLANTVILLAGE_DIR = f'{DRIVE_ROOT}/aclis_ready_plantvillage_dataset'

DATASET_DIR = '/content/leaf_pest_others_dataset'

OUTPUT_DIR      = f'{DRIVE_ROOT}/pest_gate_output'
SAVE_PATH       = os.path.join(OUTPUT_DIR, 'aclis_pest_gate_96x.pth')
EMA_PATH        = os.path.join(OUTPUT_DIR, 'aclis_pest_gate_96x_ema.pth')
TEACHER_PATH    = os.path.join(OUTPUT_DIR, 'pest_gate_teacher_mnv3.pth')
CHECKPOINT_PATH = os.path.join(OUTPUT_DIR, 'aclis_pest_gate_96x_checkpoint.pth')
TFLITE_PATH     = os.path.join(OUTPUT_DIR, 'aclis_pest_gate_96x_full_int8.tflite')

IMAGE_SIZE  = 96
NUM_CLASSES = 3
# ImageFolder alphabetical order — keep this exact list
CLASSES     = ['leaf', 'others', 'pest']
BATCH_SIZE  = 64
TARGET_ACC  = 0.90

# Recipe hyperparams (same as leaf-gate alt)
TEACHER_EPOCHS   = 8
STUDENT_EPOCHS   = 40
LR_TEACHER       = 3e-4
LR_STUDENT       = 1e-3
WEIGHT_DECAY     = 5e-4
WARMUP_EPOCHS    = 3
KD_TEMPERATURE   = 4.0
KD_ALPHA         = 0.7
LABEL_SMOOTH     = 0.05
MIXUP_ALPHA      = 0.2
```

```
EMA_DECAY        = 0.999
QAT_EPOCHS       = 5
PATIENCE         = 12

SKIP_IF_DATASET_EXISTS = True

os.makedirs(OUTPUT_DIR, exist_ok=True)

print('Pest Gate config ready')
print(f'  DATASET_DIR     : {DATASET_DIR}')
print(f'  OUTPUT_DIR      : {OUTPUT_DIR}')
print(f'  TFLITE_PATH     : {TFLITE_PATH}')
print(f'  Classes         : {CLASSES}  (NUM_CLASSES={NUM_CLASSES})')
print(f'  Student epochs  : {STUDENT_EPOCHS}  KD a={KD_ALPHA}  T={KD_TEMPERATURE}')
print(f'  MixUp a={MIXUP_ALPHA}  EMA={EMA_DECAY}  QAT epochs={QAT_EPOCHS}')
```

```
Pest Gate config ready
  DATASET_DIR     : /content/leaf_pest_others_dataset
  OUTPUT_DIR      : /content/drive/MyDrive/pest_gate_output
  TFLITE_PATH     : /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_full_int8.tflite
  Classes         : ['leaf', 'others', 'pest']  (NUM_CLASSES=3)
  Student epochs  : 40  KD a=0.7  T=4.0
  MixUp a=0.2  EMA=0.999  QAT epochs=5
```

2 — Build / load leaf · pest · others dataset

If `leaf_pest_others_dataset.zip` is on Drive, extract it. Otherwise assemble from:

1. `leaf_noleaf_dataset.zip` → `leaf` + `others` (`Non_Leaf`)
2. `aclis_ready_plantvillage_dataset/{split}/pest` → `pest`

[6] 1m

```
import os
import shutil
import time
import zipfile
from pathlib import Path

print('Dataset cell started', flush=True)

IMG_EXTS = {'.jpg', '.jpeg', '.png', '.bmp', '.webp', '.gif', '.tif', '.tiff'}

def dataset_ready(root):
    for split in ('train', 'val', 'test'):
        for cls in CLASSES:
            d = os.path.join(root, split, cls)
            if not os.path.isdir(d) or not any(Path(d).iterdir()):
                return False
    return True

def summarize(root):
    for split in ('train', 'val', 'test'):
        counts = {cls: len(os.listdir(os.path.join(root, split, cls))) for cls in CLASSES}
        print(f'  [{split}] ' + '  '.join(f'{k}={v}' for k, v in counts.items()), flush=True)

def is_image(p: Path) -> bool:
    return p.is_file() and p.suffix.lower() in IMG_EXTS

def copy_images(src_dir: Path, dst_dir: Path):
    dst_dir.mkdir(parents=True, exist_ok=True)
    n = 0
    for f in src_dir.iterdir():
        if not is_image(f):
            continue
        name = f.name.replace(' ', '_')
        dst = dst_dir / name
        if dst.exists():
            stem, suf = dst.stem, dst.suffix
            k = 1
            while dst.exists():
                dst = dst_dir / f'{stem}_{k}{suf}'
                k += 1
        shutil.copy2(f, dst)
        n += 1
    return n

def find_leaf_split_root(extract_root):
    for root, dirs, _ in os.walk(extract_root):
        if 'train' not in dirs:
            continue
        children = set(os.listdir(os.path.join(root, 'train')))
        if {'Leaf', 'Non_Leaf'} <= children or {'leaf', 'not_leaf'} <= children:
            return root
        if any(c in children for c in ('Leaf', 'Non_Leaf', 'NonLeaf', 'noleaf', 'leaf')):
            return root
    return None

def build_from_sources(dst_root):
    if os.path.isdir(dst_root):
        shutil.rmtree(dst_root)

    if not os.path.isfile(LEAF_NOLEAF_ZIP):
        raise FileNotFoundError(
            f'Need either {PREBUILT_ZIP} or {LEAF_NOLEAF_ZIP} on Drive.'
        )
    extract_tmp = '/content/_leaf_noleaf_extract'
    if os.path.isdir(extract_tmp):
        shutil.rmtree(extract_tmp)
    os.makedirs(extract_tmp, exist_ok=True)
    print(f'  Unzipping leaf/noleaf: {LEAF_NOLEAF_ZIP}', flush=True)
    with zipfile.ZipFile(LEAF_NOLEAF_ZIP, 'r') as zf:
        zf.extractall(extract_tmp)
    leaf_root = find_leaf_split_root(extract_tmp)
    if leaf_root is None:
        raise RuntimeError(f'Could not find train/Leaf in {extract_tmp}')
    print(f'  Detected leaf_noleaf root: {leaf_root}', flush=True)

    if not os.path.isdir(os.path.join(PLANTVILLAGE_DIR, 'train', 'pest')):
        raise FileNotFoundError(
            f'PlantVillage pest folder not found under {PLANTVILLAGE_DIR}/train/pest'
        )

    leaf_aliases = {
        'leaf': 'leaf', 'Leaf': 'leaf', 'LEAF': 'leaf',
        'not_leaf': 'others', 'Non_Leaf': 'others', 'non_leaf': 'others',
        'NonLeaf': 'others', 'noleaf': 'others', 'NoLeaf': 'others',
        'non-leaf': 'others', 'others': 'others',
    }

    for split in ('train', 'val', 'test'):
        split_src = os.path.join(leaf_root, split)
        for name in os.listdir(split_src):
            src_cls = Path(split_src) / name
            if not src_cls.is_dir():
                continue
            canon = leaf_aliases.get(name)
```

```
                if canon is None:
                    key = name.lower().replace('-', '_').replace(' ', '_')
                    if key == 'leaf':
                        canon = 'leaf'
                    elif 'non' in key or 'no_leaf' in key or key in ('noleaf', 'others'):
                        canon = 'others'
                    else:
                        print(f'  skipping unknown leaf_noleaf folder: {split}/{name}', flush=True)
                        continue
                n = copy_images(src_cls, Path(dst_root) / split / canon)
                print(f'   {split}/{canon}: +{n} from leaf_noleaf/{name}', flush=True)

        pest_src = Path(PLANTVILLAGE_DIR) / split / 'pest'
        n = copy_images(pest_src, Path(dst_root) / split / 'pest')
        print(f'   {split}/pest: +{n} from plantvillage', flush=True)

    shutil.rmtree(extract_tmp, ignore_errors=True)
    if not dataset_ready(dst_root):
        raise RuntimeError(f'Build failed — check folders under {dst_root}')

if SKIP_IF_DATASET_EXISTS and dataset_ready(DATASET_DIR):
    print(f'Using existing {DATASET_DIR}', flush=True)
    summarize(DATASET_DIR)
else:
    t0 = time.time()
    if os.path.isfile(PREBUILT_ZIP):
        print(f'Found prebuilt zip: {PREBUILT_ZIP}', flush=True)
        local_zip = '/content/leaf_pest_others_dataset.zip'
        if not os.path.isfile(local_zip) or os.path.getsize(local_zip) != os.path.getsize(PREBUILT_ZIP):
            print('   Copying zip to /content...', flush=True)
            shutil.copy2(PREBUILT_ZIP, local_zip)
        extract_tmp = '/content/_pest_gate_extract'
        if os.path.isdir(extract_tmp):
            shutil.rmtree(extract_tmp)
        os.makedirs(extract_tmp, exist_ok=True)
        print('   Unzipping...', flush=True)
        with zipfile.ZipFile(local_zip, 'r') as zf:
            zf.extractall(extract_tmp)
        src_root = None
        for root, dirs, _ in os.walk(extract_tmp):
            if 'train' in dirs:
                children = set(os.listdir(os.path.join(root, 'train')))
                if set(CLASSES) <= children:
                    src_root = root
                    break
        if src_root is None:
            raise RuntimeError(f'Could not find train/{CLASSES} in zip')
        if os.path.isdir(DATASET_DIR):
            shutil.rmtree(DATASET_DIR)
        shutil.move(src_root, DATASET_DIR)
        shutil.rmtree(extract_tmp, ignore_errors=True)
    else:
        print('Prebuilt zip not found — assembling from leaf_noleaf + plantvillage pest', flush=True)
        build_from_sources(DATASET_DIR)

    print(f'Dataset ready at {DATASET_DIR} in {time.time()-t0:.0f}s', flush=True)
    summarize(DATASET_DIR)

from torchvision.datasets import ImageFolder
_probe = ImageFolder(os.path.join(DATASET_DIR, 'train'))
print('ImageFolder classes:', _probe.classes)
assert _probe.classes == CLASSES, f'Expected {CLASSES}, got {_probe.classes}'
print('Class index OK: leaf=0, others=1, pest=2', flush=True)
```

```
Dataset cell started
Found prebuilt zip: /content/drive/MyDrive/leaf_pest_others_dataset.zip
   Copying zip to /content...
   Unzipping...
Dataset ready at /content/leaf_pest_others_dataset in 71s
  [train] leaf=9065  others=8066  pest=3009
  [val] leaf=1942  others=1728  pest=644
  [test] leaf=1943  others=1730  pest=646
ImageFolder classes: ['leaf', 'others', 'pest']
Class index OK: leaf=0, others=1, pest=2
```

3 — Models: TinyPestGate student + MobileNetV3-Small teacher

Same depthwise-separable topology as Leaf Gate (MCU-friendly). Head outputs **3** logits.

[7] 0s

```
print('Model cell started', flush=True)

import copy
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models

print(f'  torch {torch.__version__}  cuda={torch.cuda.is_available()}', flush=True)

class DepthwiseSeparable(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.dw = nn.Conv2d(cin, cin, 3, stride=stride, padding=1, groups=cin, bias=False)
        self.bn1 = nn.BatchNorm2d(cin)
        self.pw = nn.Conv2d(cin, cout, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)

    def forward(self, x):
        x = F.relu6(self.bn1(self.dw(x)))
        x = F.relu6(self.bn2(self.pw(x)))
        return x

class TinyPestGate(nn.Module):
    """Same topology as TinyLeafGate — 3-class head for leaf / others / pest."""
    def __init__(self, num_classes=3):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU6(inplace=True),
        )
        self.blocks = nn.Sequential(
            DepthwiseSeparable(32, 48, stride=2),
            DepthwiseSeparable(48, 64, stride=2),
            DepthwiseSeparable(64, 96, stride=2),
            DepthwiseSeparable(96, 128, stride=1),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        x = self.stem(x)
```

```
        x = self.blocks(x)
        return self.head(x)


def build_teacher(num_classes=3):
    """MobileNetV3-Small, ImageNet weights, classifier replaced for 3-way task."""
    try:
        weights = models.MobileNet_V3_Small_Weights.IMAGENET1K_V1
        m = models.mobilenet_v3_small(weights=weights)
    except Exception:
        m = models.mobilenet_v3_small(pretrained=True)
    in_f = m.classifier[-1].in_features
    m.classifier[-1] = nn.Linear(in_f, num_classes)
    return m


DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
student = TinyPestGate(NUM_CLASSES).to(DEVICE)
teacher = build_teacher(NUM_CLASSES).to(DEVICE)

n_params = sum(p.numel() for p in student.parameters())
print('=' * 65)
print('Pest Gate — TinyPestGate student + MobileNetV3-Small teacher')
print('=' * 65)
print(f'Device            : {DEVICE}')
print(f'Student params    : {n_params:,}')
print(f'Teacher params    : {sum(p.numel() for p in teacher.parameters()):,}')
print(f'Input             : {IMAGE_SIZE}x{IMAGE_SIZE}')
print(f'Classes           : {CLASSES}')
print('=' * 65)
```

```
Model cell started
  torch 2.11.0+cu128  cuda=True
Downloading: "https://download.pytorch.org/models/mobilenet_v3_small-047dcff4.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v3_small-047dcff4.pth
100%|██████████| 9.83M/9.83M [00:00<00:00, 60.6MB/s]
=================================================================
Pest Gate — TinyPestGate student + MobileNetV3-Small teacher
=================================================================
Device            : cuda
Student params    : 27,667
Teacher params    : 1,520,931
Input             : 96x96
Classes           : ['leaf', 'others', 'pest']
=================================================================
```

4 — Dataloaders (field / camera-hardened augments)

Adds **Gaussian blur**, **JPEG compression**, and **stronger lighting swings** to approximate OV2640 capture.

[8] ✓ 0s

```
import io
import random
import numpy as np
from PIL import Image, ImageFilter
from torch.utils.data import DataLoader, WeightedRandomSampler
from torchvision import transforms
from torchvision.datasets import ImageFolder

class SafeImageFolder(ImageFolder):
    def __getitem__(self, index):
        for offset in range(5):
            idx = (index + offset) % len(self.samples)
            path, target = self.samples[idx]
            try:
                img = Image.open(path).convert('RGB')
                if self.transform:
                    img = self.transform(img)
                return img, target
            except Exception:
                continue
        return torch.zeros(3, IMAGE_SIZE, IMAGE_SIZE), 0


class RandomGaussianBlur:
    def __init__(self, p=0.35, radius=(0.3, 1.8)):
        self.p = p
        self.radius = radius
    def __call__(self, img):
        if random.random() > self.p:
            return img
        r = random.uniform(*self.radius)
        return img.filter(ImageFilter.GaussianBlur(radius=r))


class RandomJPEG:
    """Simulate camera / MCU JPEG artifacts."""
    def __init__(self, p=0.4, quality=(35, 90)):
        self.p = p
        self.quality = quality
    def __call__(self, img):
        if random.random() > self.p:
            return img
        q = random.randint(*self.quality)
        buf = io.BytesIO()
        img.save(buf, format='JPEG', quality=q)
        buf.seek(0)
        return Image.open(buf).convert('RGB')


IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.4, 1.0), ratio=(0.85, 1.15)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.RandomRotation(degrees=35),
    transforms.ColorJitter(brightness=0.55, contrast=0.55, saturation=0.4, hue=0.12),
    transforms.RandomAffine(degrees=0, translate=(0.12, 0.12), scale=(0.85, 1.15)),
    RandomGaussianBlur(p=0.35),
    RandomJPEG(p=0.4),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    transforms.RandomErasing(p=0.2, scale=(0.02, 0.12)),
])

class FixedCameraStress:
    """Deterministic field degradation so baseline vs alt see identical inputs."""
    def __init__(self):
        self.resize = transforms.Resize((IMAGE_SIZE, IMAGE_SIZE))
        self.to_tensor = transforms.ToTensor()
        self.norm = transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)

    def __call__(self, img):
        img = self.resize(img)
        # mild blur
        img = img.filter(ImageFilter.GaussianBlur(radius=0.9))
```

```
            # slightly darker / lower contrast (outdoor underexposure proxy)
            arr = np.asarray(img).astype(np.float32)
            arr = (arr - 127.5) * 0.82 + 127.5 - 12.0
            arr = np.clip(arr, 0, 255).astype(np.uint8)
            img = Image.fromarray(arr)
            # fixed mid-quality JPEG
            buf = io.BytesIO()
            img.save(buf, format='JPEG', quality=55)
            buf.seek(0)
            img = Image.open(buf).convert('RGB')
            x = self.to_tensor(img)
            return self.norm(x)

    stress_transform = FixedCameraStress()

    val_transform = transforms.Compose([
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    ])

    train_dataset  = SafeImageFolder(os.path.join(DATASET_DIR, 'train'), transform=train_transform)
    val_dataset    = SafeImageFolder(os.path.join(DATASET_DIR, 'val'),   transform=val_transform)
    test_dataset   = SafeImageFolder(os.path.join(DATASET_DIR, 'test'),  transform=val_transform)
    # Separate folder instance so stress transform does not touch clean test
    stress_dataset = SafeImageFolder(os.path.join(DATASET_DIR, 'test'),  transform=stress_transform)

    assert train_dataset.classes == CLASSES

    counts = [0] * NUM_CLASSES
    for _, y in train_dataset.samples:
        counts[y] += 1
    print('Train counts:', dict(zip(CLASSES, counts)))

    weights = [1.0 / counts[y] for _, y in train_dataset.samples]
    sampler = WeightedRandomSampler(weights, num_samples=len(weights), replacement=True)

    train_loader  = DataLoader(train_dataset,  batch_size=BATCH_SIZE, sampler=sampler,
                               num_workers=2, pin_memory=True)
    val_loader    = DataLoader(val_dataset,    batch_size=BATCH_SIZE, shuffle=False,
                               num_workers=2, pin_memory=True)
    test_loader   = DataLoader(test_dataset,   batch_size=BATCH_SIZE, shuffle=False,
                               num_workers=2, pin_memory=True)
    stress_loader = DataLoader(stress_dataset, batch_size=BATCH_SIZE, shuffle=False,
                               num_workers=2, pin_memory=True)

    class_weights = torch.tensor(
        [sum(counts) / (NUM_CLASSES * c) for c in counts],
        dtype=torch.float32, device=DEVICE,
    )
    print('Class weights:', {c: float(w) for c, w in zip(CLASSES, class_weights)})
    print('✅ Loaders ready (train + clean val/test + camera-stress test)')
```

```
Train counts: {'leaf': 9065, 'others': 8066, 'pest': 3009}
Class weights: {'leaf': 0.7405773401260376, 'others': 0.8323001861572266, 'pest': 2.2310845851898193}
✅ Loaders ready (train + clean val/test + camera-stress test)
```

5a — Train teacher (MobileNetV3-Small)

Short fine-tune on the 3-class leaf / others / pest task. Soft logits become the distillation target for the student.

[9] ✓ 16m

```
print('▶ Teacher training started', flush=True)

import math

def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    class_correct = [0] * NUM_CLASSES
    class_total = [0] * NUM_CLASSES
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * len(images)
            preds = outputs.argmax(1)
            correct += (preds == labels).sum().item()
            total += len(images)
            for p, l in zip(preds, labels):
                class_total[l.item()] += 1
                class_correct[l.item()] += int(p.item() == l.item())
    per_class = {
        CLASSES[i]: (class_correct[i] / class_total[i] if class_total[i] else 0.0)
        for i in range(NUM_CLASSES)
    }
    return total_loss / total, correct / total, per_class

def per_class_str(per_class):
    return ' '.join(f'{c}={per_class[c]:.3f}' for c in CLASSES)

teacher_crit = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=LABEL_SMOOTH)
opt_t = torch.optim.AdamW(teacher.parameters(), lr=LR_TEACHER, weight_decay=WEIGHT_DECAY)
sched_t = torch.optim.lr_scheduler.CosineAnnealingLR(opt_t, T_max=TEACHER_EPOCHS, eta_min=1e-6)

best_t_acc = 0.0
print('=' * 65)
print(f'Teacher fine-tune  epochs={TEACHER_EPOCHS}  lr={LR_TEACHER}')
print('=' * 65)

for epoch in range(TEACHER_EPOCHS):
    teacher.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        opt_t.zero_grad()
        outputs = teacher(images)
        loss = teacher_crit(outputs, labels)
        loss.backward()
        opt_t.step()
        total_loss += loss.item() * len(images)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += len(images)
    sched_t.step()
    tr_acc = correct / total
    _, va_acc, per_class = evaluate(teacher, val_loader, teacher_crit)
    marker = ''
    if va_acc > best_t_acc:
        best_t_acc = va_acc
        torch.save(teacher.state_dict(), TEACHER_PATH)
        marker = ' ← saved'
    print(f'  Epoch {epoch+1:3d}/{TEACHER_EPOCHS} | train {tr_acc:.3f} | val {va_acc:.3f} | '
          f'{per_class_str(per_class)}{marker}')

teacher.load_state_dict(torch.load(TEACHER_PATH, map_location=DEVICE))
teacher.eval()
```

```
for p in teacher.parameters():
    p.requires_grad_(False)
print(f'\n✅ Teacher ready  best val={best_t_acc:.4f}  → {TEACHER_PATH}')
```

```
▶ Teacher training started
============================================================
Teacher fine-tune  epochs=8  lr=0.0003
============================================================
  Epoch   1/8 | train 0.844 | val 0.971 | leaf=0.963 others=0.975 pest=0.981 ← saved
  Epoch   2/8 | train 0.927 | val 0.983 | leaf=0.982 others=0.983 pest=0.988 ← saved
  Epoch   3/8 | train 0.945 | val 0.991 | leaf=0.996 others=0.988 pest=0.984 ← saved
  Epoch   4/8 | train 0.955 | val 0.990 | leaf=0.989 others=0.989 pest=0.997
  Epoch   5/8 | train 0.960 | val 0.990 | leaf=0.991 others=0.987 pest=0.997
  Epoch   6/8 | train 0.965 | val 0.991 | leaf=0.991 others=0.989 pest=0.997 ← saved
  Epoch   7/8 | train 0.968 | val 0.991 | leaf=0.993 others=0.988 pest=0.995 ← saved
  Epoch   8/8 | train 0.967 | val 0.992 | leaf=0.994 others=0.989 pest=0.995 ← saved

✅ Teacher ready  best val=0.9921  → /content/drive/MyDrive/pest_gate_output/pest_gate_teacher_mnv3.pth
```

5b — Distill student (KD + MixUp + EMA + warmup-cosine)

Hard loss: label-smoothed CE (with MixUp). Soft loss: KL between teacher/student softened logits.

[10] ✓ 1h

```
print('▶ Student distillation started', flush=True)

class ModelEMA:
    def __init__(self, model, decay=0.999):
        self.ema = copy.deepcopy(model).eval()
        for p in self.ema.parameters():
            p.requires_grad_(False)
        self.decay = decay
    @torch.no_grad()
    def update(self, model):
        d = self.decay
        msd = model.state_dict()
        for k, v in self.ema.state_dict().items():
            if v.dtype.is_floating_point:
                v.mul_(d).add_(msd[k].detach(), alpha=1.0 - d)
            else:
                v.copy_(msd[k])


def mixup_batch(x, y, alpha):
    if alpha <= 0:
        return x, y, y, 1.0
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1.0 - lam) * x[idx], y, y[idx], lam


def kd_loss(student_logits, teacher_logits, y_a, y_b, lam, class_w):
    # hard (mixup) CE
    log_p = F.log_softmax(student_logits, dim=1)
    ce_a = F.nll_loss(log_p, y_a, weight=class_w, reduction='none')
    ce_b = F.nll_loss(log_p, y_b, weight=class_w, reduction='none')
    hard = (lam * ce_a + (1.0 - lam) * ce_b).mean()
    # soft KD
    T = KD_TEMPERATURE
    soft = F.kl_div(
        F.log_softmax(student_logits / T, dim=1),
        F.softmax(teacher_logits / T, dim=1),
        reduction='batchmean',
    ) * (T * T)
    return KD_ALPHA * soft + (1.0 - KD_ALPHA) * hard


def lr_at_epoch(epoch):
    """Linear warmup then cosine decay."""
    if epoch < WARMUP_EPOCHS:
        return LR_STUDENT * float(epoch + 1) / float(WARMUP_EPOCHS)
    progress = (epoch - WARMUP_EPOCHS) / max(1, STUDENT_EPOCHS - WARMUP_EPOCHS)
    return 1e-6 + 0.5 * (LR_STUDENT - 1e-6) * (1.0 + math.cos(math.pi * progress))


opt_s = torch.optim.AdamW(student.parameters(), lr=LR_STUDENT, weight_decay=WEIGHT_DECAY)
ema = ModelEMA(student, decay=EMA_DECAY)
ce_eval = nn.CrossEntropyLoss(weight=class_weights)

best_val_acc = 0.0
patience_counter = 0

print('=' * 65)
print(f'Student KD  epochs={STUDENT_EPOCHS}  lr={LR_STUDENT}  α_KD={KD_ALPHA}  MixUp={MIXUP_ALPHA}')
print('=' * 65)

for epoch in range(STUDENT_EPOCHS):
    lr = lr_at_epoch(epoch)
    for g in opt_s.param_groups:
        g['lr'] = lr

    student.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        mixed, y_a, y_b, lam = mixup_batch(images, labels, MIXUP_ALPHA)
        opt_s.zero_grad()
        s_logits = student(mixed)
        with torch.no_grad():
            t_logits = teacher(mixed)
        loss = kd_loss(s_logits, t_logits, y_a, y_b, lam, class_weights)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(student.parameters(), 5.0)
        opt_s.step()
        ema.update(student)

        total_loss += loss.item() * len(images)
        # accuracy on non-mixed labels is noisy under MixUp — use raw student preds vs y_a
        correct += (s_logits.argmax(1) == y_a).sum().item()
        total += len(images)

    tr_acc = correct / total
    _, va_acc, per_class = evaluate(ema.ema, val_loader, ce_eval)
    marker = ''
    if va_acc > best_val_acc:
        best_val_acc = va_acc
        patience_counter = 0
        torch.save(student.state_dict(), SAVE_PATH)
        torch.save(ema.ema.state_dict(), EMA_PATH)
        marker = ' ← saved EMA'
    else:
        patience_counter += 1
        marker = f' (patience {patience_counter}/{PATIENCE})'

    torch.save({
        'epoch': epoch,
        'student': student.state_dict(),
```

```
            'ema': ema.ema.state_dict(),
            'best_val_acc': best_val_acc,
        }, CHECKPOINT_PATH)

        print(f'  Epoch {epoch+1:3d}/{STUDENT_EPOCHS} | lr {lr:.2e} | train~{tr_acc:.3f} | '
              f'val(EMA) {va_acc:.3f} | {per_class_str(per_class)}{marker}')

        if patience_counter >= PATIENCE:
            print(f'\n  Early stopping at epoch {epoch+1}')
            break

# Prefer EMA for eval / export
student.load_state_dict(torch.load(EMA_PATH, map_location=DEVICE))
print(f'\nBest EMA val accuracy: {best_val_acc:.4f}')
print(f'Loaded EMA weights from {EMA_PATH}')
```

```
▶ Student distillation started
================================================================
Student KD  epochs=40  lr=0.001  α_KD=0.7  MixUp=0.2
================================================================
  Epoch   1/40 | lr 3.33e-04 | train~0.433 | val(EMA) 0.149 | leaf=0.000 others=0.000 pest=1.000 ← saved EMA
  Epoch   2/40 | lr 6.67e-04 | train~0.515 | val(EMA) 0.149 | leaf=0.000 others=0.000 pest=1.000 (patience 1/12)
  Epoch   3/40 | lr 1.00e-03 | train~0.548 | val(EMA) 0.149 | leaf=0.000 others=0.000 pest=1.000 (patience 2/12)
  Epoch   4/40 | lr 1.00e-03 | train~0.564 | val(EMA) 0.149 | leaf=0.000 others=0.000 pest=1.000 (patience 3/12)
  Epoch   5/40 | lr 9.98e-04 | train~0.556 | val(EMA) 0.149 | leaf=0.000 others=0.000 pest=1.000 (patience 4/12)
  Epoch   6/40 | lr 9.93e-04 | train~0.566 | val(EMA) 0.149 | leaf=0.000 others=0.000 pest=1.000 (patience 5/12)
  Epoch   7/40 | lr 9.84e-04 | train~0.573 | val(EMA) 0.246 | leaf=0.214 others=0.001 pest=1.000 ← saved EMA
  Epoch   8/40 | lr 9.71e-04 | train~0.557 | val(EMA) 0.478 | leaf=0.677 others=0.067 pest=0.983 ← saved EMA
  Epoch   9/40 | lr 9.56e-04 | train~0.577 | val(EMA) 0.648 | leaf=0.837 others=0.323 pest=0.952 ← saved EMA
  Epoch  10/40 | lr 9.37e-04 | train~0.574 | val(EMA) 0.756 | leaf=0.880 others=0.550 pest=0.935 ← saved EMA
  Epoch  11/40 | lr 9.14e-04 | train~0.586 | val(EMA) 0.832 | leaf=0.902 others=0.718 pest=0.927 ← saved EMA
  Epoch  12/40 | lr 8.89e-04 | train~0.594 | val(EMA) 0.869 | leaf=0.911 others=0.800 pest=0.925 ← saved EMA
  Epoch  13/40 | lr 8.61e-04 | train~0.584 | val(EMA) 0.892 | leaf=0.915 others=0.850 pest=0.933 ← saved EMA
  Epoch  14/40 | lr 8.31e-04 | train~0.605 | val(EMA) 0.906 | leaf=0.916 others=0.883 pest=0.938 ← saved EMA
  Epoch  15/40 | lr 7.98e-04 | train~0.583 | val(EMA) 0.916 | leaf=0.918 others=0.902 pest=0.946 ← saved EMA
  Epoch  16/40 | lr 7.62e-04 | train~0.602 | val(EMA) 0.923 | leaf=0.922 others=0.914 pest=0.947 ← saved EMA
  Epoch  17/40 | lr 7.25e-04 | train~0.596 | val(EMA) 0.927 | leaf=0.922 others=0.924 pest=0.952 ← saved EMA
  Epoch  18/40 | lr 6.87e-04 | train~0.589 | val(EMA) 0.931 | leaf=0.924 others=0.931 pest=0.950 ← saved EMA
  Epoch  19/40 | lr 6.47e-04 | train~0.590 | val(EMA) 0.935 | leaf=0.927 others=0.937 pest=0.953 ← saved EMA
  Epoch  20/40 | lr 6.06e-04 | train~0.614 | val(EMA) 0.936 | leaf=0.928 others=0.938 pest=0.953 ← saved EMA
  Epoch  21/40 | lr 5.64e-04 | train~0.590 | val(EMA) 0.938 | leaf=0.932 others=0.940 pest=0.955 ← saved EMA
  Epoch  22/40 | lr 5.22e-04 | train~0.602 | val(EMA) 0.941 | leaf=0.935 others=0.942 pest=0.957 ← saved EMA
  Epoch  23/40 | lr 4.79e-04 | train~0.590 | val(EMA) 0.943 | leaf=0.938 others=0.942 pest=0.958 ← saved EMA
  Epoch  24/40 | lr 4.37e-04 | train~0.608 | val(EMA) 0.943 | leaf=0.939 others=0.942 pest=0.960 ← saved EMA
  Epoch  25/40 | lr 3.95e-04 | train~0.614 | val(EMA) 0.945 | leaf=0.940 others=0.944 pest=0.964 ← saved EMA
  Epoch  26/40 | lr 3.54e-04 | train~0.612 | val(EMA) 0.946 | leaf=0.942 others=0.944 pest=0.964 ← saved EMA
  Epoch  27/40 | lr 3.14e-04 | train~0.591 | val(EMA) 0.949 | leaf=0.946 others=0.947 pest=0.966 ← saved EMA
  Epoch  28/40 | lr 2.76e-04 | train~0.632 | val(EMA) 0.951 | leaf=0.945 others=0.950 pest=0.969 ← saved EMA
  Epoch  29/40 | lr 2.39e-04 | train~0.591 | val(EMA) 0.950 | leaf=0.943 others=0.950 pest=0.969 (patience 1/12)
  Epoch  30/40 | lr 2.03e-04 | train~0.591 | val(EMA) 0.950 | leaf=0.943 others=0.951 pest=0.969 (patience 2/12)
  Epoch  31/40 | lr 1.70e-04 | train~0.617 | val(EMA) 0.951 | leaf=0.944 others=0.953 pest=0.970 ← saved EMA
  Epoch  32/40 | lr 1.40e-04 | train~0.632 | val(EMA) 0.952 | leaf=0.944 others=0.953 pest=0.970 ← saved EMA
  Epoch  33/40 | lr 1.12e-04 | train~0.620 | val(EMA) 0.952 | leaf=0.944 others=0.954 pest=0.972 ← saved EMA
  Epoch  34/40 | lr 8.67e-05 | train~0.604 | val(EMA) 0.952 | leaf=0.944 others=0.954 pest=0.972 (patience 1/12)
  Epoch  35/40 | lr 6.44e-05 | train~0.586 | val(EMA) 0.953 | leaf=0.944 others=0.955 pest=0.974 ← saved EMA
  Epoch  36/40 | lr 4.53e-05 | train~0.599 | val(EMA) 0.953 | leaf=0.944 others=0.955 pest=0.975 ← saved EMA
  Epoch  37/40 | lr 2.95e-05 | train~0.600 | val(EMA) 0.953 | leaf=0.944 others=0.955 pest=0.978 ← saved EMA
  Epoch  38/40 | lr 1.71e-05 | train~0.622 | val(EMA) 0.954 | leaf=0.944 others=0.955 pest=0.978 ← saved EMA
  Epoch  39/40 | lr 8.18e-06 | train~0.606 | val(EMA) 0.954 | leaf=0.945 others=0.955 pest=0.978 ← saved EMA
  Epoch  40/40 | lr 2.80e-06 | train~0.621 | val(EMA) 0.955 | leaf=0.946 others=0.957 pest=0.978 ← saved EMA

Best EMA val accuracy: 0.9550
Loaded EMA weights from /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_ema.pth
```

5c — Quantization-aware training (fake-quant)

A few epochs of QAT so the float graph is already INT8-aware before TFLite conversion. Falls back gracefully if `torch.ao.quantization` APIs differ.

[11] ✓ 10m

```
print('▶ QAT fine-tune started', flush=True)

# Lightweight fake-quant via torch.quantization (eager) on the student.
# If QAT setup fails on this Colab stack, we skip and rely on PTQ (still valid).

qat_ok = False
try:
    import torch.ao.quantization as tq

    student.cpu()
    student.train()
    student.qconfig = tq.get_default_qat_qconfig('qnnpack')
    # Fuse nothing fancy — TinyLeafGate has ReLU6 after BN which fuses poorly in eager.
    # Instead: train with FakeQuant inserted on activations/weights via prepare_qat on a wrapper.
    # Simpler robust approach used below: noise-injection QAT proxy (additive uniform noise
    # scaled like INT8 bins) — works across torch versions without fuse requirements.
    raise RuntimeError('use noise-proxy QAT')
except Exception as e:
    print(f'  Eager QAT path skipped ({e}); using INT8-noise proxy QAT.', flush=True)

def int8_noise_forward(module, x):
    """Fake-quant activations to 256 bins with straight-through estimator."""
    if x.dim() == 4:
        xmin = x.amin(dim=(0, 2, 3), keepdim=True)
        xmax = x.amax(dim=(0, 2, 3), keepdim=True)
    else:
        xmin = x.amin(dim=0, keepdim=True)
        xmax = x.amax(dim=0, keepdim=True)
    scale = (xmax - xmin).clamp_min(1e-6) / 255.0
    x_q = torch.round((x - xmin) / scale) * scale + xmin
    # STE: forward = quantized, backward ≈ identity through x
    return x + (x_q - x).detach()

class QATProxy(nn.Module):
    def __init__(self, base):
        super().__init__()
        self.base = base
    def forward(self, x):
        # inject noise after stem and before head
        x = self.base.stem(x)
        if self.training:
            x = int8_noise_forward(self, x)
        x = self.base.blocks(x)
        if self.training:
            x = int8_noise_forward(self, x)
        return self.base.head(x)

student = student.to(DEVICE)
qat_model = QATProxy(student).to(DEVICE)
opt_q = torch.optim.AdamW(qat_model.parameters(), lr=1e-4, weight_decay=WEIGHT_DECAY)
ce_q = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=LABEL_SMOOTH)

print('=' * 65)
print(f'QAT-proxy fine-tune  epochs={QAT_EPOCHS}  lr=1e-4')
print('=' * 65)

for epoch in range(QAT_EPOCHS):
    qat_model.train()
    total_loss, correct, total = 0.0, 0, 0
```

```
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        opt_q.zero_grad()
        logits = qat_model(images)
        with torch.no_grad():
            t_logits = teacher(images)
        # keep a bit of KD during QAT
        hard = ce_q(logits, labels)
        soft = F.kl_div(
            F.log_softmax(logits / KD_TEMPERATURE, dim=1),
            F.softmax(t_logits / KD_TEMPERATURE, dim=1),
            reduction='batchmean',
        ) * (KD_TEMPERATURE ** 2)
        loss = 0.5 * soft + 0.5 * hard
        loss.backward()
        opt_q.step()
        total_loss += loss.item() * len(images)
        correct += (logits.argmax(1) == labels).sum().item()
        total += len(images)
    tr_acc = correct / total
    # eval without noise
    student.eval()
    _, va_acc, per_class = evaluate(student, val_loader, ce_eval)
    print(f'  QAT {epoch+1:3d}/{QAT_EPOCHS} | train {tr_acc:.3f} | val {va_acc:.3f} | '
          f'{per_class_str(per_class)}')

torch.save(student.state_dict(), EMA_PATH)
torch.save(student.state_dict(), SAVE_PATH)
print(f'✅ QAT-proxy done — weights saved to {EMA_PATH}')
```

```
▶ QAT fine-tune started
  Eager QAT path skipped (use noise-proxy QAT); using INT8-noise proxy QAT.
================================================================
QAT-proxy fine-tune  epochs=5  lr=1e-4
================================================================
  QAT   1/5 | train 0.878 | val 0.956 | leaf=0.951 others=0.957 pest=0.969
  QAT   2/5 | train 0.882 | val 0.950 | leaf=0.950 others=0.942 pest=0.977
  QAT   3/5 | train 0.883 | val 0.959 | leaf=0.956 others=0.959 pest=0.966
  QAT   4/5 | train 0.882 | val 0.954 | leaf=0.953 others=0.948 pest=0.974
  QAT   5/5 | train 0.878 | val 0.955 | leaf=0.951 others=0.951 pest=0.974
✅ QAT-proxy done — weights saved to /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_ema.pth
```

6 — Test evaluation (PyTorch FP32)

Clean test **and** camera-stress test (blur + lighting).

[12] ✓ 34s

```
student.load_state_dict(torch.load(EMA_PATH, map_location=DEVICE))
student.to(DEVICE).eval()

te_loss, te_acc, per_class = evaluate(student, test_loader, ce_eval)
st_loss, st_acc, st_per = evaluate(student, stress_loader, ce_eval)

print('=' * 65)
print('PyTorch FP32 TEST')
print('=' * 65)
print(f'  Clean accuracy       : {te_acc:.4f} ({te_acc*100:.1f}%)')
print(f'  Camera-stress acc    : {st_acc:.4f} ({st_acc*100:.1f}%)')
for cls in CLASSES:
    print(f'    clean {cls:<8} {per_class[cls]:.3f}   stress {st_per[cls]:.3f}')
print(f'  Target               : {TARGET_ACC:.2f}')
print('=' * 65)
```

```
============================================================
PyTorch FP32 TEST
============================================================
  Clean accuracy       : 0.9558 (95.6%)
  Camera-stress acc    : 0.9090 (90.9%)
    clean leaf     0.957   stress 0.911
    clean others   0.953   stress 0.891
    clean pest     0.960   stress 0.952
  Target               : 0.90
============================================================
```

7 — Export TinyEngine-friendly INT8 TFLite

Keras twin with GAP(keepdims)+1×1 Conv so TinyEngine codegen still works.

[13] ✓ 9s

```
print('▶ TinyEngine-friendly export started', flush=True)

import numpy as np
import tensorflow as tf

student.load_state_dict(torch.load(EMA_PATH, map_location='cpu'))
student.cpu().eval()
sd = student.state_dict()
model = student  # alias used by export / eval cells

def pt_conv_to_keras(w):
    return w.detach().cpu().numpy().transpose(2, 3, 1, 0)

def pt_dw_to_keras(w):
    return w.detach().cpu().numpy().transpose(2, 3, 0, 1)

def pt_linear_to_conv1x1(w):
    arr = w.detach().cpu().numpy().T
    return arr.reshape(1, 1, arr.shape[0], arr.shape[1])

def keras_bn_weights(prefix):
    return [
        sd[f'{prefix}.weight'].detach().cpu().numpy(),
        sd[f'{prefix}.bias'].detach().cpu().numpy(),
        sd[f'{prefix}.running_mean'].detach().cpu().numpy(),
        sd[f'{prefix}.running_var'].detach().cpu().numpy(),
    ]

def pt_matched_bn(name):
    return tf.keras.layers.BatchNormalization(epsilon=1e-5, momentum=0.9, name=name)

def conv_pt_pad(x, layer, name):
    x = tf.keras.layers.ZeroPadding2D(1, name=f'{name}_pad')(x)
    return layer(x)

def build_keras_twin():
    def ds_block(x, cout, stride, name):
        x = conv_pt_pad(
            x,
            tf.keras.layers.DepthwiseConv2D(
                3, strides=stride, padding='valid', use_bias=False, name=f'{name}_dw'),
            f'{name}_dw',
        )
        x = pt_matched_bn(f'{name}_bn1')(x)
        x = tf.keras.layers.ReLU(max_value=6.0, name=f'{name}_relu1')(x)
        x = tf.keras.layers.Conv2D(cout, 1, use_bias=False, name=f'{name}_pw')(x)
```

```
            x = pt_matched_bn(f'{name}_bn2')(x)
            x = tf.keras.layers.ReLU(max_value=6.0, name=f'{name}_relu2')(x)
            return x

        inp = tf.keras.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3), name='input')
        x = conv_pt_pad(
            inp,
            tf.keras.layers.Conv2D(
                32, 3, strides=2, padding='valid', use_bias=False, name='stem_conv'),
            'stem',
        )
        x = pt_matched_bn('stem_bn')(x)
        x = tf.keras.layers.ReLU(max_value=6.0, name='stem_relu')(x)
        x = ds_block(x, 48, 2, 'b0')
        x = ds_block(x, 64, 2, 'b1')
        x = ds_block(x, 96, 2, 'b2')
        x = ds_block(x, 128, 1, 'b3')
        x = tf.keras.layers.GlobalAveragePooling2D(keepdims=True, name='gap')(x)
        x = tf.keras.layers.Conv2D(NUM_CLASSES, 1, use_bias=True, name='cls_conv')(x)
        out = tf.keras.layers.Reshape((NUM_CLASSES,), name='output')(x)
        return tf.keras.Model(inp, out, name='TinyPestGate_TE')

    kmodel = build_keras_twin()
    kmodel.get_layer('stem_conv').set_weights([pt_conv_to_keras(sd['stem.0.weight'])])
    kmodel.get_layer('stem_bn').set_weights(keras_bn_weights('stem.1'))

    for kname, ptname in [('b0', 'blocks.0'), ('b1', 'blocks.1'),
                          ('b2', 'blocks.2'), ('b3', 'blocks.3')]:
        kmodel.get_layer(f'{kname}_dw').set_weights([pt_dw_to_keras(sd[f'{ptname}.dw.weight'])])
        kmodel.get_layer(f'{kname}_bn1').set_weights(keras_bn_weights(f'{ptname}.bn1'))
        kmodel.get_layer(f'{kname}_pw').set_weights([pt_conv_to_keras(sd[f'{ptname}.pw.weight'])])
        kmodel.get_layer(f'{kname}_bn2').set_weights(keras_bn_weights(f'{ptname}.bn2'))

    kmodel.get_layer('cls_conv').set_weights([
        pt_linear_to_conv1x1(sd['head.3.weight']),
        sd['head.3.bias'].detach().cpu().numpy(),
    ])

    print('✅ Keras twin built and weights copied')
    kmodel.summary()

    kmodel.trainable = False
    n_check = min(16, len(val_dataset))
    diffs = []
    for i in range(n_check):
        img, _ = val_dataset[i]
        with torch.no_grad():
            pt_out = model(img.unsqueeze(0)).numpy().reshape(-1)
        x = img.numpy().transpose(1, 2, 0)[None, ...].astype(np.float32)
        k_out = kmodel.predict(x, verbose=0).reshape(-1)
        diffs.append(np.max(np.abs(pt_out - k_out)))

    print(f'Float max|PT-Keras| over {n_check} images: mean={np.mean(diffs):.5f}  max={np.max(diffs):.5f}')
    if np.max(diffs) > 0.01:
        raise RuntimeError(
            f'PT↔Keras logit mismatch too large (max={np.max(diffs):.5f}). Do not export.'
        )
    print('✅ PT ↔ Keras logits match')
```

b0_relu1 (ReLU)	(None, 24, 24, 32)	0
b0_pw (Conv2D)	(None, 24, 24, 48)	1,536
b0_bn2 (BatchNormalization)	(None, 24, 24, 48)	192
b0_relu2 (ReLU)	(None, 24, 24, 48)	0
b1_dw_pad (ZeroPadding2D)	(None, 26, 26, 48)	0
b1_dw (DepthwiseConv2D)	(None, 12, 12, 48)	432
b1_bn1 (BatchNormalization)	(None, 12, 12, 48)	192
b1_relu1 (ReLU)	(None, 12, 12, 48)	0
b1_pw (Conv2D)	(None, 12, 12, 64)	3,072
b1_bn2 (BatchNormalization)	(None, 12, 12, 64)	256
b1_relu2 (ReLU)	(None, 12, 12, 64)	0
b2_dw_pad (ZeroPadding2D)	(None, 14, 14, 64)	0
b2_dw (DepthwiseConv2D)	(None, 6, 6, 64)	576
b2_bn1 (BatchNormalization)	(None, 6, 6, 64)	256
b2_relu1 (ReLU)	(None, 6, 6, 64)	0
b2_pw (Conv2D)	(None, 6, 6, 96)	6,144
b2_bn2 (BatchNormalization)	(None, 6, 6, 96)	384
b2_relu2 (ReLU)	(None, 6, 6, 96)	0
b3_dw_pad (ZeroPadding2D)	(None, 8, 8, 96)	0
b3_dw (DepthwiseConv2D)	(None, 6, 6, 96)	864
b3_bn1 (BatchNormalization)	(None, 6, 6, 96)	384
b3_relu1 (ReLU)	(None, 6, 6, 96)	0
b3_pw (Conv2D)	(None, 6, 6, 128)	12,288
b3_bn2 (BatchNormalization)	(None, 6, 6, 128)	512
b3_relu2 (ReLU)	(None, 6, 6, 128)	0
gap (GlobalAveragePooling2D)	(None, 1, 1, 128)	0
cls_conv (Conv2D)	(None, 1, 1, 3)	387
output (Reshape)	(None, 3)	0

Total params: 28,883 (112.82 KB)
Trainable params: 27,667 (108.07 KB)
Non-trainable params: 1,216 (4.75 KB)

```
Float max|PT-Keras| over 16 images: mean=0.00000  max=0.00000
✅ PT ↔ Keras logits match
```

[14] ✓ 4s

```
print('▶ INT8 TFLite conversion', flush=True)

calib_ds = val_dataset

def make_rep_data():
    n = min(200, len(calib_ds))
    for i in range(n):
        img, _ = calib_ds[i]
        x = img.numpy().transpose(1, 2, 0)[None, ...].astype(np.float32)
        yield [x]

TF_SAVED = '/content/aclis_leaf_gate_alt_tf'
!rm -rf {TF_SAVED}
kmodel.export(TF_SAVED)
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(kmodel)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = make_rep_data
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_model = converter.convert()

os.makedirs(os.path.dirname(TFLITE_PATH) or '.', exist_ok=True)
with open(TFLITE_PATH, 'wb') as f:
    f.write(tflite_model)

tflite_kb = os.path.getsize(TFLITE_PATH) / 1024.0
print(f'✅ INT8 TFLite saved: {TFLITE_PATH}')
print(f'   File size: {tflite_kb:.1f} KB')
```

```
Saved artifact at '/tmp/tmp3k6yqgp1'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 96, 96, 3), dtype=tf.float32, name='input')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  138058694060432: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694064656: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694063888: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694062736: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694064272: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694061968: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694060240: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694064464: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694064080: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694065040: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058694065232: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424846864: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424847056: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424845520: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424846096: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424847440: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424847632: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424846672: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424847248: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424848208: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424845904: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424849360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424849552: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424848784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424845712: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424848400: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424848016: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424847824: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424848976: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424850512: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424849936: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424850128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424850320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424849168: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424851472: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424850896: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424851088: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424851280: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424845136: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424852432: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424852816: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424853008: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424852048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424852624: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424853584: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424853968: TensorSpec(shape=(), dtype=tf.resource, name=None)
  138058424853776: TensorSpec(shape=(), dtype=tf.resource, name=None)
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/convert.py:863: UserWarning: Statistics for quantized inputs were expected, but not specified; continuing anyway.
  warnings.warn(
✅ INT8 TFLite saved: /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_full_int8.tflite
   File size: 51.0 KB
```

[15]
✓ 0s

```
print('▶ Validate TFLite graph for TinyEngine', flush=True)

from tensorflow.lite.python import schema_py_generated as schema_fb

interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
interp.allocate_tensors()
inp = interp.get_input_details()[0]
out = interp.get_output_details()[0]
print('Input :', list(inp['shape']), inp['dtype'], 'quant=', inp.get('quantization'))
print('Output:', list(out['shape']), out['dtype'], 'quant=', out.get('quantization'))

_OP_NAMES = {
    getattr(schema_fb.BuiltinOperator, k): k
    for k in dir(schema_fb.BuiltinOperator)
    if k.isupper() and isinstance(getattr(schema_fb.BuiltinOperator, k), int)
}

buf = open(TFLITE_PATH, 'rb').read()
fb_model = schema_fb.Model.GetRootAsModel(buf, 0)
g = fb_model.Subgraphs(0)

ops, bad = [], []
for i in range(g.OperatorsLength()):
    op = g.Operators(i)
    oc = fb_model.OperatorCodes(op.OpcodeIndex())
    code = oc.BuiltinCode()
    if code == 0 and hasattr(oc, 'DeprecatedBuiltinCode'):
        try:
            dep = oc.DeprecatedBuiltinCode()
            if dep:
                code = dep
        except Exception:
            pass
    name = _OP_NAMES.get(code, f'OP_{code}')
    ops.append(name)
    if name == 'RESHAPE':
        out_ti = op.Outputs(0)
        t = g.Tensors(out_ti)
        shape = [int(t.Shape(j)) for j in range(t.ShapeLength())]
        if shape in ([1, 1], [1]):
            bad.append(f'RESHAPE → {shape}')
        else:
            print(f'  reshape ok: {shape}')

print('\\nOperators:')
for i, n in enumerate(ops):
    print(f'  {i:02d} {n}')

has_pool = any(n in ('MEAN', 'AVERAGE_POOL_2D') for n in ops)
has_cls = any(n in ('CONV_2D', 'FULLY_CONNECTED') for n in ops)

x = np.zeros(tuple(inp['shape']), dtype=np.int8)
interp.set_tensor(inp['index'], x)
interp.invoke()
print('Smoke output:', interp.get_tensor(out['index']))

if bad or not has_pool or not has_cls:
    raise RuntimeError(f'Graph not TinyEngine-friendly: bad={bad} pool={has_pool} cls={has_cls}')
print('\\n✅ TFLite looks TinyEngine-friendly')
```

```
print('Download:', TFLITE_PATH)
```

```
▶ Validate TFLite graph for TinyEngine
Input : [np.int32(1), np.int32(96), np.int32(96), np.int32(3)] <class 'numpy.int8'> quant= (0.01845697872340679, -16)
Output: [np.int32(1), np.int32(3)] <class 'numpy.int8'> quant= (0.029870159924030304, -11)
  reshape ok: [1, 3]
\nOperators:
  00 PAD
  01 CONV_2D
  02 PAD
  03 DEPTHWISE_CONV_2D
  04 CONV_2D
  05 PAD
  06 DEPTHWISE_CONV_2D
  07 CONV_2D
  08 PAD
  09 DEPTHWISE_CONV_2D
  10 CONV_2D
  11 DEPTHWISE_CONV_2D
  12 CONV_2D
  13 MEAN
  14 CONV_2D
  15 SHAPE
  16 STRIDED_SLICE
  17 PACK
  18 RESHAPE
Smoke output: [[-44  15   0]]
\n✅ TFLite looks TinyEngine-friendly
Download: /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_full_int8.tflite
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:     Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

7b — INT8 TFLite test

[16] ✓ 1m

```
print('▶ INT8 TFLite full test-set evaluation (ALT)', flush=True)

interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
interp.allocate_tensors()
inp = interp.get_input_details()[0]
out = interp.get_output_details()[0]
in_scale, in_zp = inp['quantization']
assert in_scale > 0

def quantize_nhwc(chw_float: np.ndarray) -> np.ndarray:
    x = chw_float.transpose(1, 2, 0).astype(np.float32)
    v = x / float(in_scale)
    v = v + np.where(v >= 0.0, 0.5, -0.5)
    q = np.trunc(v).astype(np.int32) + int(in_zp)
    return np.clip(q, -128, 127).astype(np.int8)

def eval_tflite_on_dataset(ds, tag):
    correct = 0
    total = 0
    class_correct = [0] * NUM_CLASSES
    class_total = [0] * NUM_CLASSES
    agree_pt = 0
    model.cpu().eval()
    for i in range(len(ds)):
        img, label = ds[i]
        x_int8 = quantize_nhwc(img.numpy())
        interp.set_tensor(inp['index'], x_int8[None, ...])
        interp.invoke()
        y = interp.get_tensor(out['index']).reshape(-1)
        pred = int(np.argmax(y))
        with torch.no_grad():
            pt_pred = int(model(img.unsqueeze(0)).argmax(1).item())
        correct += int(pred == label)
        agree_pt += int(pred == pt_pred)
        total += 1
        class_total[label] += 1
        class_correct[label] += int(pred == label)
        if (i + 1) % 500 == 0 or (i + 1) == total:
            print(f'  [{tag}] … {i+1}/{len(ds)}', flush=True)
    acc = correct / total
    per = {CLASSES[i]: (class_correct[i] / class_total[i] if class_total[i] else 0.0)
           for i in range(NUM_CLASSES)}
    return acc, agree_pt / total, per

int8_acc, pt_agree, int8_per = eval_tflite_on_dataset(test_dataset, 'clean')
int8_stress, _, int8_stress_per = eval_tflite_on_dataset(stress_dataset, 'stress')
gap = abs(int8_acc - te_acc)

print('=' * 65)
print('Pest Gate INT8 TFLite TEST')
print('=' * 65)
print(f'  Clean INT8 accuracy  : {int8_acc:.4f} ({int8_acc*100:.1f}%)')
print(f'  Clean PyTorch FP32   : {te_acc:.4f} ({te_acc*100:.1f}%)')
print(f'  |Δ| accuracy         : {gap:.4f}')
print(f'  Pred agreement PT    : {pt_agree:.4f}')
print(f'  Camera-stress INT8   : {int8_stress:.4f} ({int8_stress*100:.1f}%)')
for cls in CLASSES:
    print(f'    clean {cls:<8} {int8_per[cls]:.3f}   stress {int8_stress_per[cls]:.3f}')
print(f'  Artifact             : {TFLITE_PATH}')
print('=' * 65)

if gap > 0.01:
    raise RuntimeError(
        f'INT8 test acc ({int8_acc:.4f}) differs from PyTorch ({te_acc:.4f}) by {gap:.4f} > 1%.'
    )
print('✅ INT8 ↔ PyTorch clean test accuracy within 1%')
```

```
  [stress] … 4276/4319
  [stress] … 4277/4319
  [stress] … 4278/4319
  [stress] … 4279/4319
  [stress] … 4280/4319
  [stress] … 4281/4319
  [stress] … 4282/4319
  [stress] … 4283/4319
  [stress] … 4284/4319
  [stress] … 4285/4319
  [stress] … 4286/4319
  [stress] … 4287/4319
  [stress] … 4288/4319
  [stress] … 4289/4319
  [stress] … 4290/4319
  [stress] … 4291/4319
  [stress] … 4292/4319
  [stress] … 4293/4319
  [stress] … 4294/4319
  [stress] … 4295/4319
  [stress] … 4296/4319
  [stress] … 4297/4319
  [stress] … 4298/4319
  [stress] … 4299/4319
  [stress] … 4300/4319
```

```
  [stress] … 4301/4319
  [stress] … 4302/4319
  [stress] … 4303/4319
  [stress] … 4304/4319
  [stress] … 4305/4319
  [stress] … 4306/4319
  [stress] … 4307/4319
  [stress] … 4308/4319
  [stress] … 4309/4319
  [stress] … 4310/4319
  [stress] … 4311/4319
  [stress] … 4312/4319
  [stress] … 4313/4319
  [stress] … 4314/4319
  [stress] … 4315/4319
  [stress] … 4316/4319
  [stress] … 4317/4319
  [stress] … 4318/4319
  [stress] … 4319/4319
============================================================
Pest Gate INT8 TFLite TEST
============================================================
  Clean INT8 accuracy  : 0.9558 (95.6%)
  Clean PyTorch FP32   : 0.9558 (95.6%)
  |Δ| accuracy        : 0.0000
  Pred agreement PT    : 0.9984
  Camera-stress INT8   : 0.9106 (91.1%)
    clean leaf     0.956   stress 0.915
    clean others   0.954   stress 0.890
    clean pest     0.961   stress 0.954
  Artifact             : /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_full_int8.tflite
============================================================
✅ INT8 ↔ PyTorch clean test accuracy within 1%
```

8 — Confusion matrix + flash budget

Per-class confusion on clean INT8 test, plus STM32 flash picture.

[17] ✓ 21s

```python
print('Confusion / flash summary', flush=True)

import numpy as np

interp = tf.lite.Interpreter(model_path=TFLITE_PATH)
interp.allocate_tensors()
ind = interp.get_input_details()[0]
oud = interp.get_output_details()[0]
scale, zp = ind['quantization']

cm = np.zeros((NUM_CLASSES, NUM_CLASSES), dtype=np.int64)
for i in range(len(test_dataset)):
    img, label = test_dataset[i]
    x = img.numpy().transpose(1, 2, 0).astype(np.float32)
    v = x / float(scale)
    v = v + np.where(v >= 0.0, 0.5, -0.5)
    q = np.clip(np.trunc(v).astype(np.int32) + int(zp), -128, 127).astype(np.int8)
    interp.set_tensor(ind['index'], q[None, ...])
    interp.invoke()
    pred = int(np.argmax(interp.get_tensor(oud['index']).reshape(-1)))
    cm[label, pred] += 1
    if (i + 1) % 500 == 0 or (i + 1) == len(test_dataset):
        print(f'  … {i+1}/{len(test_dataset)}', flush=True)

print('Confusion matrix (rows=true, cols=pred):')
header = 'true\\pred'.ljust(12) + ''.join(f'{c:>10}' for c in CLASSES)
print(header)
for i, cls in enumerate(CLASSES):
    row = f'{cls:<12}' + ''.join(f'{cm[i, j]:10d}' for j in range(NUM_CLASSES))
    print(row)

pest_kb = os.path.getsize(TFLITE_PATH) / 1024.0
DISEASE_FLASH = 723
FLASH_BUDGET  = 1024
print()
print('=' * 65)
print('Flash picture (weights only)')
print('=' * 65)
print(f'  Disease model     : {DISEASE_FLASH} KB')
print(f'  Pest Gate         : {pest_kb:.1f} KB')
print(f'  Disease + Gate    : {DISEASE_FLASH + pest_kb:.1f} / {FLASH_BUDGET} KB  '
      f'({"FITS" if DISEASE_FLASH + pest_kb <= FLASH_BUDGET else "OVER"})')
print('=' * 65)
print(f'\nArtifact: {TFLITE_PATH}')
print('Classes :', CLASSES)
```

```
Confusion / flash summary
  … 500/4319
  … 1000/4319
  … 1500/4319
  … 2000/4319
  … 2500/4319
  … 3000/4319
  … 3500/4319
  … 4000/4319
  … 4319/4319
Confusion matrix (rows=true, cols=pred):
true\pred         leaf    others      pest
leaf              1857        11        75
others              28      1650        52
pest                 7        18       621

============================================================
Flash picture (weights only)
============================================================
  Disease model     : 723 KB
  Pest Gate         : 51.0 KB
  Disease + Gate    : 774.0 / 1024 KB  (FITS)
============================================================

Artifact: /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_full_int8.tflite
Classes : ['leaf', 'others', 'pest']
```

9 — Next steps

1. Upload `leaf_pest_others_dataset/` (or zip it) to Drive if training on Colab, **or** upload `leaf_noleaf_dataset.zip` + `aclis_ready_plantvillage_dataset/` and let Section 2 assemble it.
2. Run this notebook end-to-end on a GPU Colab runtime.
3. Copy `aclis_pest_gate_96x_full_int8.tflite` into the STM32 / TinyEngine codegen path when ready to cascade after (or instead of) the binary leaf gate.

[18] ✓ 0s

```python
print('Pest Gate notebook complete.')
print(f'  Dataset : {DATASET_DIR}')
print(f'  Classes : {CLASSES}')
print(f'  TFLite  : {TFLITE_PATH}')
if os.path.isfile(TFLITE_PATH):
    print(f'  Size    : {os.path.getsize(TFLITE_PATH)/1024:.1f} KB')
```

```
Pest Gate notebook complete.
  Dataset : /content/leaf_pest_others_dataset
  Classes : ['leaf', 'others', 'pest']
  TFLite  : /content/drive/MyDrive/pest_gate_output/aclis_pest_gate_96x_full_int8.tflite
  Size    : 51.0 KB
```